Output and algebraic classification of phonological processes

Anton Hampe
anton.hampe@yale.edu
11/07/26

Yale University

The basic problem

Algebraic approaches are nice because they give us a very fine grained typology and lots of existing results, they often let us test for membership in classes efficiently and they offer a unified representational scheme for processes and languages. Algebraic and automata-theoretic approaches also come easier to me personally than logical ones.

The basic problem

Algebraic approaches are nice because they give us a very fine grained typology and lots of existing results, they often let us test for membership in classes efficiently and they offer a unified representational scheme for processes and languages. Algebraic and automata-theoretic approaches also come easier to me personally than logical ones.

But despite the unified scheme, our current methods for algebraic approaches to classifying processes largely ignore output. I think it would be really nice to develop a classification scheme that takes output into account to a greater degree.

Constraints and Processes

Phonotactics

What kind of combinations of speech sounds are permissible in a given language?

A phonotactic grammar specifies a formal language.

German satisfies $*[-son, +voi]\#$

Processes

How does the pronounciation of phonemes or morphemes change depending on environment?

A phonological grammar specifies a function from strings to strings.

$[-son, +voi] \rightarrow [-voi] / __ \#$

Ying and Yang

Phonological processes and phonotactic constraints are not independent of each other.

Ying and Yang

Phonological processes and phonotactic constraints are not independent of each other.

The fact that final devoicing exists as a process is the reason that the corresponding phonotactic constraint holds in German. The fact that the phonotactic constraint holds is what motivated the rule in the first place.

Ying and Yang

Phonological processes and phonotactic constraints are not independent of each other.

The fact that final devoicing exists as a process is the reason that the corresponding phonotactic constraint holds in German. The fact that the phonotactic constraint holds is what motivated the rule in the first place.

A big part of the appeal of OT is bringing these two aspects of phonology together in a way that previous types of grammars often didn't.

Ying and Yang

Phonological processes and phonotactic constraints are not independent of each other.

The fact that final devoicing exists as a process is the reason that the corresponding phonotactic constraint holds in German. The fact that the phonotactic constraint holds is what motivated the rule in the first place.

A big part of the appeal of OT is bringing these two aspects of phonology together in a way that previous types of grammars often didn't.

Since we have a nice algebraic framework for treating transductions and languages, it would be nice if we had a single algebraic structure that reflected both the input behavior of a phonological map and the phonotactics it generates.

Output languages

A transduction has a strictly local output language if it maps its domain to a strictly local language.

Output languages

A transduction has a strictly local output language if it maps its domain to a strictly local language.

Final devoicing $\{+voice, -sonorant\} \rightarrow \{-voice\}/__\#$ has an SL output language.

Output languages

A transduction has a strictly local output language if it maps its domain to a strictly local language.

Final devoicing $\{+voice, -sonorant\} \rightarrow \{-voice\}/__\#$ has an SL output language.

Chandlee et al. (2014) show that this is not a trivial property. Belonging to the ISL class, OSL class, or both is not enough to ensure that the output language of a transduction is strictly local.

Output languages of ISL rules

Do actual local phonological processes have SL output languages?

Output languages of ISL rules

Do actual local phonological processes have SL output languages?

Out of the rules with context length up to 2 on either side, the following have non-SL output languages under simultaneous application.

Output languages of ISL rules

Do actual local phonological processes have SL output languages?

Out of the rules with context length up to 2 on either side, the following have non-SL output languages under simultaneous application.

$\emptyset \to x / x__x$	$\emptyset \to x / x__xx$	$\emptyset \to x / x__$	$\emptyset \to x / yy__$
$\emptyset \to x / xy__$	$\emptyset \to x / xx__x$	$\emptyset \to x / xx__xx$	$\emptyset \to x / xx__$
$\emptyset \to x / __x$	$\emptyset \to x / __yy$	$\emptyset \to x / __yx$	$\emptyset \to x / __xx$

Are such rules attested?

PBase is a large database of phonological rules.

Are such rules attested?

PBase is a large database of phonological rules.

I searched PBase for ISL rules with output languages that are not SL and practically none are attested. Of the few cases in the database, most are transparent transcription errors.

Behavior under composition

Having an SL output language is not necessarily preserved under composition.

Behavior under composition

Having an SL output language is not necessarily preserved under composition.

$\emptyset \rightarrow A/B__$ followed by $A \rightarrow B/B__$ is an example of the property being lost under composition.

Behavior under composition

Having an SL output language is not necessarily preserved under composition.

$\emptyset \rightarrow A / B__$ followed by $A \rightarrow B / B__$ is an example of the property being lost under composition.

The stronger property of weakly preserving SL languages is preserved under composition. A function is weakly SL preserving if the image of every SL language is an SL language.

Behavior under composition

Having an SL output language is not necessarily preserved under composition.

$\emptyset \rightarrow A / B__$ followed by $A \rightarrow B / B__$ is an example of the property being lost under composition.

The stronger property of weakly preserving SL languages is preserved under composition. A function is weakly SL preserving if the image of every SL language is an SL language.

But most local SPE rules are not weakly SL preserving.
For example: $C \rightarrow B / A__$ maps $(ABAC)^*$ to $(ABAB)^{2n}$.

Continuity

Continuity is another nice property relating processes and languages. A functions is SL continuous if the inverse image of every SL language is an SL language.

Continuity

Continuity is another nice property relating processes and languages. A functions is SL continuous if the inverse image of every SL language is an SL language.

I think this property is really nice because it essentially tells you that the set of possible underlying forms should not be more complex than the restrictions that hold of surface phonotactics.

Continuity

Continuity is another nice property relating processes and languages. A functions is SL continuous if the inverse image of every SL language is an SL language.

I think this property is really nice because it essentially tells you that the set of possible underlying forms should not be more complex than the restrictions that hold of surface phonotactics.

I am pretty sure that SL-continuity for ISL processes is implied by an existing proposed restriction: bounded deletion.

Interim summary

Classifying the input structure of a transducer and the structure of its output language gives us a finer classification of phonological processes and it lets us identify a nice universal in PBase: ISL processes have SL output languages.

Interim summary

Classifying the input structure of a transducer and the structure of its output language gives us a finer classification of phonological processes and it lets us identify a nice universal in PBase: ISL processes have SL output languages.

But we are treating these two as separate structures, even though they come from the same transduction and are usually even defined over the same alphabet!

Interim summary

Classifying the input structure of a transducer and the structure of its output language gives us a finer classification of phonological processes and it lets us identify a nice universal in PBase: ISL processes have SL output languages.

But we are treating these two as separate structures, even though they come from the same transduction and are usually even defined over the same alphabet!

Do we get a more informative structure if we combine the two?

Interim summary

Classifying the input structure of a transducer and the structure of its output language gives us a finer classification of phonological processes and it lets us identify a nice universal in PBase: ISL processes have SL output languages.

But we are treating these two as separate structures, even though they come from the same transduction and are usually even defined over the same alphabet!

Do we get a more informative structure if we combine the two?

The next couple slides give an intro/review to algebraic classification of languages and transductions and then proposes one way they could be combined.

Semigroups

A semigroup is a set of elements together with a binary operation on the elements.

Semigroups

A semigroup is a set of elements together with a binary operation on the elements.

The set is closed under the operation, multiplying any two elements in the set gives an element in the set.

Semigroups

A semigroup is a set of elements together with a binary operation on the elements.

The set is closed under the operation, multiplying any two elements in the set gives an element in the set.

It is also associative, so we can write products without parentheses.

Semigroups

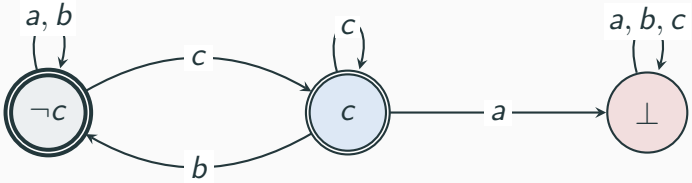
A semigroup is a set of elements together with a binary operation on the elements.

The set is closed under the operation, multiplying any two elements in the set gives an element in the set.

It is also associative, so we can write products without parentheses.

An example is the set of all strings over an alphabet under concatenation. But that is an infinitely large set! In algebraic approaches to formal languages and transductions we often manage to use finite semigroups, by treating certain strings as equivalent.

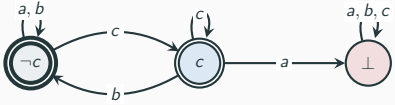
The minimal acceptor for $^*ca$

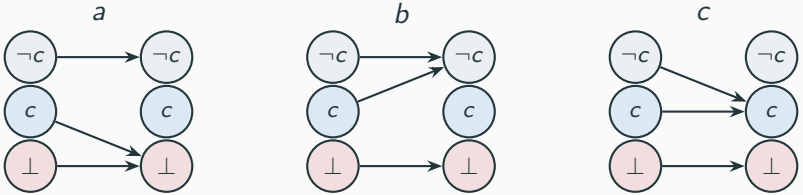


$\Sigma = \{a, b, c\}$ strings with no factor ca

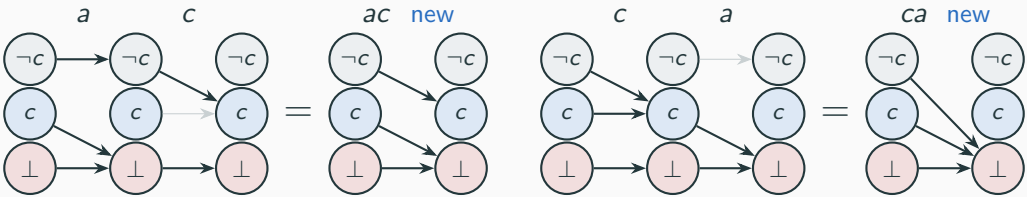
every output of $a \to b \,/\, c__$ satisfies $^*ca$

Building N: each letter is a map on $\{\neg c, c, \bot\}$

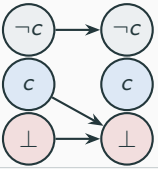
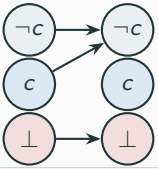
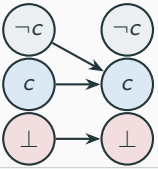
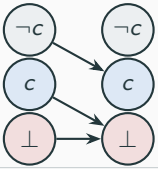
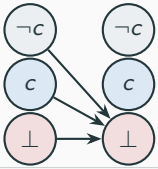




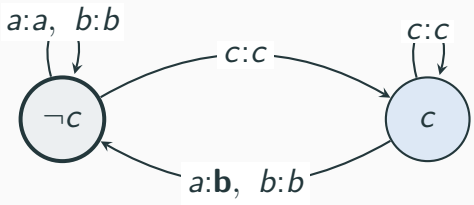
Building N: multiplying is composing maps



All elements of N

$x =$	a	b	c	ac	$ca = 0$
					
$x\,a =$	a	b	ca	ca	ca
$x\,b =$	a	b	b	a	ca
$x\,c =$	ac	c	c	ac	ca
$x\,ac =$	ac	c	ca	ca	ca
$x\,ca =$	ca	ca	ca	ca	ca

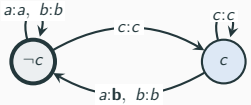
The minimal transducer for $a \to b \,/\, c __$

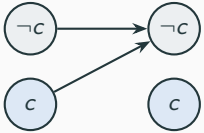
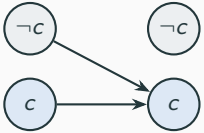


each state records whether the previous symbol was c

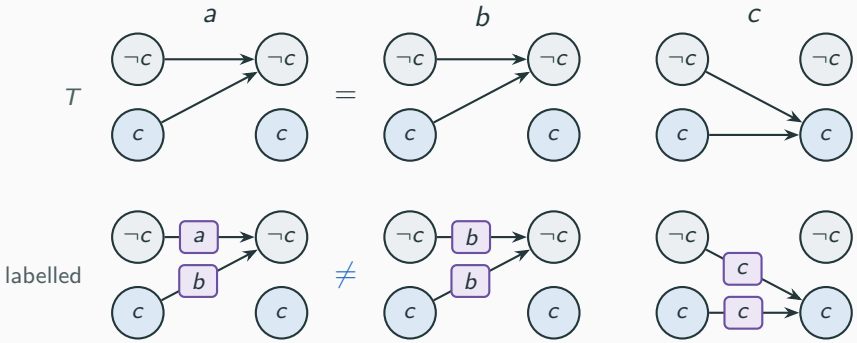
$caa \mapsto cba$ $\qquad$ $acacb \mapsto acbcb$

Building T: each letter is a map on $\{\neg c, c\}$

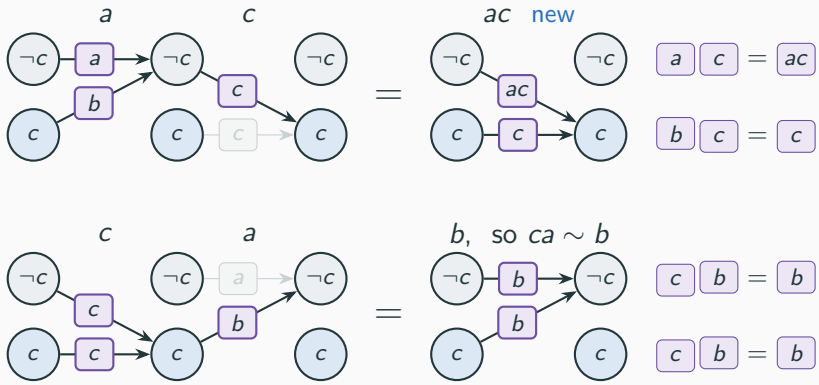


$x =$	$a = b$ 	c 
$x\,a =$	a	a
$x\,c =$	c	c

Adding labels: the output class on each arrow



Composing: follow the arrows, multiply the labels in N



Three semigroups

T

transducer

$\cdot$	a	c
a	a	c
c	a	c

N

output language ${}^*ca$

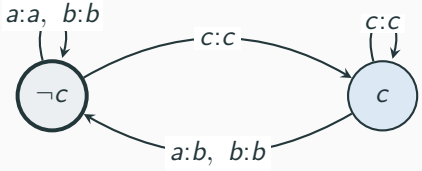
$\cdot$	a	b	c	ac	ca
a	a	a	ac	ac	ca
b	b	b	c	c	ca
c	ca	b	c	ca	ca
ac	ca	a	ac	ca	ca
ca	ca	ca	ca	ca	ca

M_f

combined

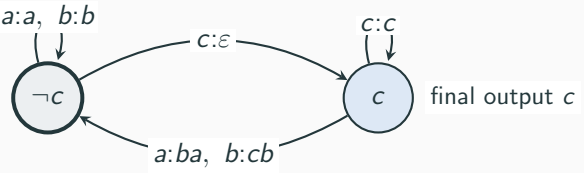
$\cdot$	a	b	c	ac
a	a	a	ac	ac
b	b	b	c	c
c	b	b	c	c
ac	a	a	ac	ac

Rule 1: $a \to b \,/\, c __$



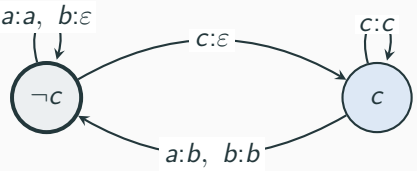
- a becomes b after c (simultaneous application)
- $caa \mapsto cba$, $cca \mapsto ccb$, $acab \mapsto acbb$, $acca \mapsto accb$

Rule 2: $c \to b \,/\, __\, a$



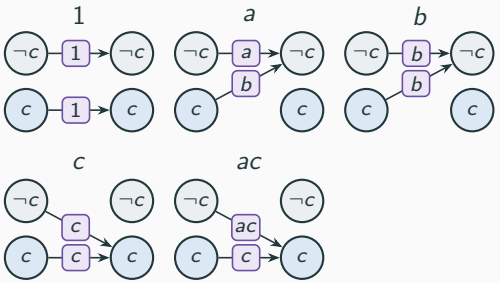
- c becomes b before a
- each c is held back (state c) until the next symbol is read; a held c at the end of the word is emitted as final output
- $caa \mapsto baa$, $cca \mapsto cba$, $acab \mapsto abab$, $acca \mapsto acba$

Rule 3: deletion except after c



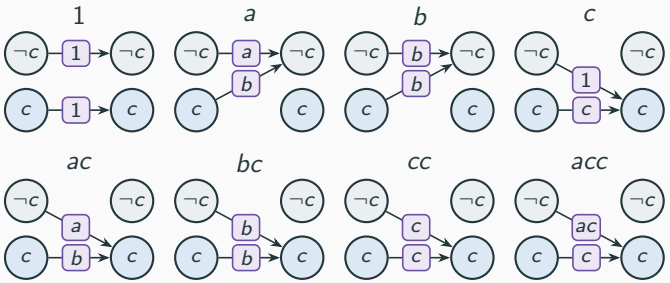
- after c: $a \to b$; b and c are kept
- elsewhere: b and c are deleted, a is kept
- the b-loop on $\neg c$ outputs ε: an ε-cycle (unbounded deletion)
- $caa \mapsto ba$, $cca \mapsto cb$, $acab \mapsto ab$, $bab \mapsto a$

Rule 1:



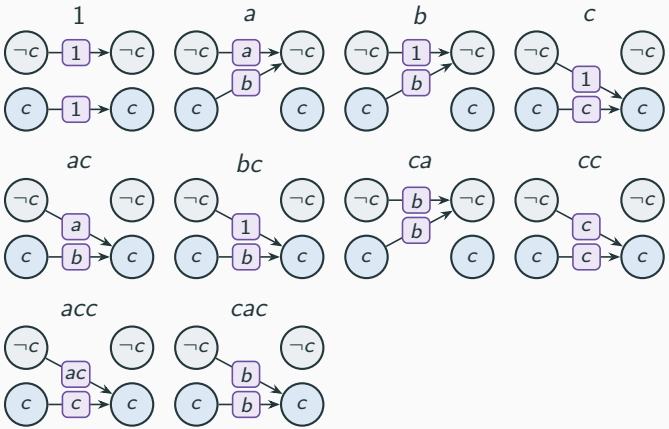
·	1	a	b	c	ac
1	1	a	b	c	ac
a	a	a	a	ac	ac
b	b	b	b	c	c
c	c	b	b	c	c
ac	ac	a	a	ac	ac

Rule 2:



$\cdot$	1	a	b	c	ac	bc	cc	acc
1	1	a	b	c	ac	bc	cc	acc
a	a	a	a	ac	ac	ac	acc	acc
b	b	b	b	bc	bc	bc	cc	cc
c	c	b	b	cc	bc	bc	cc	cc
ac	ac	a	a	acc	ac	ac	acc	acc
bc	bc	b	b	cc	bc	bc	cc	cc
cc	cc	b	b	cc	bc	bc	cc	cc
acc	acc	a	a	acc	ac	ac	acc	acc

Rule 3:



·	1	a	b	c	ac	bc	ca	cc	acc	cac
1	1	a	b	c	ac	bc	ca	cc	acc	cac
a	a	a	a	ac	ac	ac	a	acc	acc	ac
b	b	a	b	bc	ac	bc	ca	cc	acc	cac
c	c	ca	ca	cc	cac	cac	ca	cc	cc	cac
ac	ac	a	a	acc	ac	ac	a	acc	acc	ac
bc	bc	ca	ca	cc	cac	cac	ca	cc	cc	cac
ca	ca	ca	ca	cac	cac	cac	ca	cc	cc	cac
cc	cc	ca	ca	cc	cac	cac	ca	cc	cc	cac
acc	acc	a	a	acc	ac	ac	a	acc	acc	ac
cac	cac	ca	ca	cc	cac	cac	ca	cc	cc	cac

Conclusion and next steps

By combining the syntactic semigroup of the transducer with that of the output language we get a richer algebraic structure that is able to tell apart transductions that have identical transformation semigroups and output languages.

Conclusion and next steps

By combining the syntactic semigroup of the transducer with that of the output language we get a richer algebraic structure that is able to tell apart transductions that have identical transformation semigroups and output languages.

It would be interesting to look at the combined semigroups of processes in PBase and understand their complexity.

Conclusion and next steps

By combining the syntactic semigroup of the transducer with that of the output language we get a richer algebraic structure that is able to tell apart transductions that have identical transformation semigroups and output languages.

It would be interesting to look at the combined semigroups of processes in PBase and understand their complexity.

We can also try other labelings, by creating products not with the syntactic semigroup of the output language, but the one underlying the transducer, or a (direct) product of the two. That would take advantage of the fact that the input and output alphabet are the same in a way that the current product doesn't.

Conclusion and next steps

This combination of output language and transformation semigroup is a subsemigroup of their wreath product. This is the same kind of product people use in Krohn Rhodes theory to (de)compose transductions.

Conclusion and next steps

This combination of output language and transformation semigroup is a subsemigroup of their wreath product. This is the same kind of product people use in Krohn Rhodes theory to (de)compose transductions.

One of the problems with applying Krohn Rhodes theory to phonology is that their composition has no real notion of a output alphabet, rather later machines operate directly on the state of earlier machines. Perhaps we can get more natural compositions and decompositions by layering in the output languages of each process.

Thank you!

References

Chandlee, J., R. Eyraud, and J. Heinz (2014). Learning strictly local subsequential functions. *Transactions of the Association for Computational Linguistics 2*, 491–504.